

Partie 1 : SED

1. Présentation :

Sed signifie "Stream EDitor" autrement dit "éditeur de flux", et plus précisément "éditeur de flux orienté ligne". De par sa conception et son mode de fonctionnement, Sed est un éditeur non-interactif.

Tout comme l'éditeur "ed" -dont il est issu et que l'on trouve toujours dans les distributions actuelles- et contrairement aux autres éditeurs tels que vi, emacs, Nedit, Xedit, etc., qui eux fonctionnent sur une page complète de texte affiché à l'écran, Sed agit sur une seule ligne à la fois.

À ses débuts l'éditeur "ed" s'est vu doté d'une commande travaillant sur son flux d'entrée standard plutôt que sur un fichier, capable d'afficher toutes les lignes contenant une expression régulière. Cette commande dont la syntaxe s'écrit sous la forme "g/re/p" (global/regular expression/print) donnera naissance à l'utilitaire "grep". Quelques temps après une nouvelle implémentation d'une version de "ed" vit le jour, travaillant uniquement sur le flux d'entrée standard tout en tirant ses instructions d'un fichier de scripts. Cette version fut baptisée Stream EDitor, plus connue sous le nom de "Sed".

L'éditeur de flux Sed lit les lignes d'un ou plusieurs fichiers depuis l'entrée standard, enchaîne des commandes lues elles aussi depuis l'entrée standard sous forme d'expressions (commandes d'édition) ou depuis un fichier texte (script), et écrit le résultat du traitement sur la sortie standard.

On pourrait résumer le mécanisme de fonctionnement de Sed de cette façon :

1. lecture d'une ligne sur le flux d'entrée (une ligne étant délimitée par un caractère de saut de ligne) ;
2. traitement de la ligne en fonctions des diverses commandes lues ;
3. affichage (ou non) du résultat sur la sortie standard (écran) ;
4. passage à la ligne suivante.

Notons que pour sélectionner la ou les ligne(s) sur la(les)quelle(s) elles doivent opérer, les commandes acceptent des numéros de lignes, des intervalles, ou encore des expressions régulières (notées RE ou regex).

La commande sed est une commande très riche.

2. Introduction SED :

Sed est éditeur ligne non interactif, il lit les lignes d'un fichier une à une (ou provenant de l'entrée standard) leur applique un certain nombre de commandes d'édition et renvoie les lignes résultantes sur la sortie standard. Il ne modifie pas le fichier traité, il écrit tout sur la sortie standard.

Sed est une évolution de l'éditeur **ed** lui-même précurseur de **vi**, la syntaxe n'est franchement pas très conviviale, mais il permet de réaliser des commandes complexes sur des gros fichiers.

3. SED : Syntaxe générale :

La syntaxe de **sed** est la suivante :

```
sed [-n] [-e commande] [-f fichier de commandes] [fichier]
```

`[-n]` : Indique le mode silencieux.

`[-e commande]` : Spécifie les commandes à appliquer sur le fichier.

`[-f fichier de commandes]` : Indique que les commandes sont lu à partir d'un fichier.

`[fichier]` : Fichier source.

Vous disposez de l'option **-n** qui supprime la sortie standard par défaut, **sed** va écrire uniquement les lignes concernées par le traitement (sinon il écrit tout même les lignes non traitées). L'option **-e** n'est pas nécessaire quand vous avez une seule fonction d'édition.

4. SED : Adressage :

Sed peut adresser directement une ligne (*ou un intervalle*) par son numéro de ligne ou par la mise en correspondance avec une expression régulière ciblant un motif (*patron/modèle*) défini.

Un point d'exclamation (!) après un numéro de ligne, un motif ou une expression régulière exclura cette ligne (*ou intervalle*) du traitement à effectuer.

Cet adressage peut s'effectuer sous les formes suivantes :

a. Numéro :

- Le numéro de la ligne

o `sed -n 3p fich.txt`

b. Début ~ pas :

- Toutes les n lignes (*pas*) en partant de début.

o sed -n 1~2p fich.txt

c. \$:

- La dernière ligne du dernier fichier fourni en entrée, ou de chaque fichier si les options "-i" ou "-s" ont été spécifiées

o sed -n '\$ p' fich.txt
o sed -ns '\$ p' fich*
o

d. /exp/ :

- Toutes les lignes mises en correspondance avec l'expression régulière exp

o sed -n '/est/p' fich2.txt

e. , num2 :

Toutes les lignes comprises entre num1 et num2. Si num2 venait à être inférieur à num1, seul num1 est affiché

o sed -n '3,6 p' fich.txt

f. /exp1/, /exp2/ :

Toutes les lignes comprises entre exp1 et exp2, y compris les lignes contenant exp1 et exp2. Si l'intervalle contenant les 2 expressions se répète plusieurs fois, Sed appliquera les instructions à chaque intervalle successivement. Si toutefois exp2 n'est pas trouvée, les instructions sont appliquées à chaque ligne en partant de exp1 et ce jusqu'à la fin du fichier.

o sed -n '/commentaire1/,/commentaire2/ p' fich2.txt

g. num,/exp/ /exp/,num :

Toutes les lignes comprises entre un numéro de ligne et une expression régulière (ou l'inverse). Dans "num,/exp/", si toutefois exp n'est pas trouvée, les instructions sont appliquées à chaque ligne en partant de num et ce jusqu'à la fin du fichier. Dans "/exp/,num", si num est inférieur au numéro de ligne correspondant à exp, seul la ligne contenant exp est affichée.

o sed -n '2,/commentaire2/ p' fich2.txt
o sed -n '/commentaire1/,8 p' fich2.txt

5. SED : Les commandes basiques :

a. Insertion de texte avant une ligne :

[adresse] i : Permet d'insérer un texte avant la ligne correspondante à l'adresse

Exemple : Ajouter Bonjour monsieur avant la ligne 5 :

sed '5i\Bonjour monsieur\' salut.txt

b. Ajout de texte après une ligne :

[adresse] a : Ajoute un texte après la ligne

Exemple : Insérer Bonjour monsieur après la ligne 5 :

```
sed '5a\Bonjour monsieur\' salut.txt
```

c. Suppression :

La fonction de suppression **d** supprime les lignes comprises dans un intervalle donné. La syntaxe est la suivante :

```
o sed "20,30d" fichier
```

Cette commande va supprimer les lignes 20 à 30 du fichier fichier. On peut utiliser les expressions régulières :

```
sed "/yasmine/d" fichier
```

Cette commande supprime les lignes contenant la chaîne **yasmine**. Si au contraire on ne veut pas effacer les lignes contenant la chaîne **yasmine** (toutes les autres sont supprimées), on tapera :

```
sed "/yasmine/!d" fichier
```

En fait les lignes du fichier d'entrée ne sont pas supprimées, elles le sont au niveau de la sortie standard.

L'option -e permet de passer plusieurs commandes à la suite. Je pense que c'est une bonne habitude de l'ajouter de façon systématique.

Si l'on souhaite effacer toutes les lignes comprises entre la ligne 2 et la ligne 11 on procède ainsi.

Exemple :

```
o sed '2,11 d' salut.txt
```

d. Mode silencieux :

Il existe une autre façon d'utiliser SED, particulièrement intéressante. C'est l'utilisation en mode "silencieux".

SED ne doit afficher par défaut aucune ligne. Seules les lignes intéressantes seront affichées, avec l'option p (print). Pour passer en mode "silencieux", il faut utiliser l'option sed -n.

Exemple :

```
o sed -n '/Ici/p' test.txt
o
```

e. Substitution :

La fonction de substitution **s** permet de changer la première ou toutes les occurrences d'une chaîne par une autre. La syntaxe est la suivante :

`sed "s/hajar/HAJAR/" fichier` va changer la première occurrence de la chaîne **hajar** par **HAJAR** (la première chaîne **hajar** rencontrée dans le texte uniquement).

`sed "s/toto/TOTO/3" fichier` va changer la troisième occurrence de la chaîne **hajar** par **HAJAR** (la troisième chaîne **hajar** rencontrée dans le texte uniquement).

`sed "s/toto/TOTO/g" fichier` va changer toutes les occurrences de la chaîne **hajar** par **HAJAR** (toutes les chaînes **hajar** rencontrées sont changées).

`sed "s/toto/TOTO/p" fichier` en cas de remplacement la ligne concernée est affichée sur la sortie standard (uniquement en cas de substitution).

`sed "s/toto/TOTO/w resultat" fichier` en cas de substitution la ligne en entrée est inscrite dans un fichier résultat.

La fonction de substitution peut évidemment être utilisée avec une expression régulière.

`sed -e "s/[Ff]raise/FRAISE/g" fichier` substitue toutes les chaînes **Fraise** ou **fraise** par **FRAISE**.

f. Les fonctions **q**, **r** et **w** :

La fonction **q** (quit) va interrompre l'exécution de **sed**, la ligne en cours de traitement est affichée sur la sortie standard (uniquement si **-n** n'a pas été utilisée).

La fonction **r** (read) lit le contenu d'un fichier et écrit le contenu sur la sortie standard.

La fonction **w** (write) écrit la ligne sélectionnée dans un fichier.

```
sed "/^hajar/w resultat" fichier
```

Cette commande va écrire dans le fichier résultat toutes les lignes du fichier **fichier** commençant par la chaîne **hajar**.

6. SED : Les méta-caractères :

Le méta-caractère « . » remplace dans une expression régulière un caractère unique, à l'exception du caractère retour chariot (**\n**). Par exemple `chaîne.` va identifier toutes les lignes contenant la chaîne `chaîne` suivit d'un caractère quelconque unique. Si vous voulez identifier les lignes contenant la chaîne **.cshrc**, l'expression régulière correspondante est **\.cshrc**

a. Les méta-caractères **[]** :

Les méta-caractères **[]** : permettent de désigner des caractères compris dans un certain intervalle de valeur à une position déterminée d'une chaîne de caractères. Par exemple **[Ff]raise** va identifier les chaînes **Fraise** ou **fraise**, **[a-z]oualid** va identifier une chaîne de caractère commençant par une lettre minuscule (intervalle de valeur de **a** à **z**) et suivi de la chaîne **oualid** (**aoualid**, **boualid**, ..., **zoualid**).

D'une manière plus générale voici comment **[]** peuvent être utilisés.

[A-D] intervalle de **A** à **D** (**A**, **B**, **C**, **D**) par exemple **bof[A-D]** donne **bofA**, **bofB**, **bofC**, **bofD**

[2-5] intervalle de 2 à 5 (2, 3, 4, 5) par exemple **12[2-5]2** donne 1222, 1232, 1242, 1252

[2-56] intervalle de 2 à 5 et 6 (et non pas 56) (2, 3, 4, 5, 6) par exemple **12[2-56]2** donne 1222, 1232, 1242, 1252, 1262

[a-dA-D] intervalle de **a** à **d** et **A** à **D** (**a**, **b**, **c**, **d**, **A**, **B**, **C**, **D**) par exemple **z[a-dA-D]y** donne **zay**, **zby**, **zcy**, **zdy**, **zAy**, **zBy**, **zCy**, **zDy**

[1-3-] intervalle de 1 à 3 et - (1, 2, 3, -) par exemple **[1-3-]3** donne 13, 23, 33, -3

[a-cI-K1-3] intervalle de **a** à **c**, **I** à **K** et 1 à 3 (**a**, **b**, **c**, **I**, **J**, **K**, 1, 2, 3)

On peut utiliser **[]** avec un **^** pour identifier le complément de l'expression régulière. En français pour identifier l'opposé de l'expression régulière. Vous n'avez toujours pas compris ? Voici un exemple : **[^0-9] toto** identifie les lignes contenant une chaîne **toto**, le caractère juste avant ne doit pas être un chiffre (exemple **atoto**, **gtoto** mais pas **1toto**, **5toto**). Autre exemple **[^a-zA-Z]** n'importe quel caractère sauf une lettre minuscule ou majuscule. Attention à la place de **^**, si vous tapez **[1-3^]**, c'est équivalent aux caractères 1, 2, 3 et **^**.

b. Les méta-caractères **^ et **\$** :**

Le méta-caractère **^** : identifie un début de ligne. Par exemple l'expression régulière **^a** va identifier les lignes commençant par le caractère **a**.

Le méta-caractère **\$** : identifie une fin de ligne. Par exemple l'expression régulière **a\$** va identifier les lignes se terminant par le caractère **a**.

L'expression régulière **^** : **chaîne\$** identifie les lignes qui contiennent strictement la chaîne **chaîne**.

L'expression régulière **^\$** : identifie une ligne vide.

c. Le métacaractère *** :**

Le métacaractère ***** est le caractère de répétition. L'expression régulière **a*** correspond aux lignes comportant 0 ou plusieurs caractère **a**. Son utilisation est à proscrire, car toutes les lignes, même celles ne contenant pas le caractère **a**, répondent aux critères de recherche. **x*** est une source de problèmes, il vaut mieux éviter de l'employer.

L'expression régulière **aa*** correspond aux lignes comportant 1 ou plusieurs caractères **a**.

L'expression régulière **.*** correspond à n'importe quelle chaîne de caractères.

L'expression régulière **[a-z][a-z]*** va chercher les chaînes de caractères contenant 1 ou plusieurs lettres minuscules (de **a** à **z**).

L'expression régulière `[^][^]*` est équivalent à tout sauf un blanc.

d. Les métacaractères `\ ( \)` :

Pour le traitement complexe de fichier, il est utile parfois d'identifier un certain type de chaîne pour pouvoir s'en servir dans la suite du traitement comme un sous-programme. C'est le principe des sous chaînes, pour mémoriser une sous chaîne, on utilise la syntaxe `\ (expression régulière)`, cette sous chaîne sera identifié par un chiffre compris par 1 et 9 (suivant l'ordre de définition).

Par exemple `\ ([a-z][a-z]*)` est une sous chaîne identifiant les lignes contenant une ou plusieurs lettres minuscules, pour faire appel à cette sous chaîne, on pourra utiliser `\ 1`. Voir dans le paragraphe **sed** pour un exemple.

7. SED : Conclusion :

a. Avantages :

Sed possède comme avantage de pouvoir être utilisé en ligne de commande (ou via un fichier dans lequel on a mis les commandes). Il a des performances exceptionnelles pour un langage interprété. Sed ne fait aucune modification sur le fichier source.

b. Inconvénients :

Lorsqu'une commande est appliquée, c'est sur l'ensemble du fichier qu'elle va être exécutée. Il n'est pas possible de choisir de traiter telle ou telle partie du fichier. L'utilisation avancée de SED, et les problèmes de portabilité qui peuvent être rencontrés au cours de l'écriture de scripts SED. On peut aussi noter une certaine incompréhension à la relecture d'un script écrit en SED.